Documentation Algorithmique

Projet Web Scraping + Classification NLP

Master 1 IdL — Algorithmique Python S2 — 2025/2026

PARTIE 1 — WEB SCRAPING

Correction extraction du texte

Probleme identifie : le script extrayait initialement tous les <p> de la page, recuperant les menus, newsletters et listes de videos repetes sur chaque article. Le texte de chaque article etait identique (contenu du footer).

Solution : cibler la balise <article> uniquement, qui contient le vrai corps de l'article.

XPath corrige :

```
# AVANT : extrait tout (menus, newsletters, videos...) paragraphes =
tree.xpath("//p//text()") # APRES : extrait uniquement le corps de l'article
paragraphes = tree.xpath("//article//p//text()")
```

Structures de donnees — Partie 1

Structure	Type Python	Description
Page HTML parsee	lxml.html.HtmlElement	Arbre DOM de la page
Metadonnees article	dict	{titre, date, chapeau, label}
Texte extrait	str	Corps de l'article (balise <article>)
Un article	dict	{url, titre, date, chapeau, texte, label}
Dataset complet	list[dict]	50 articles, 5 categories
Fichier de sortie	CSV	6 colonnes, encodage UTF-8

Fonctions principales

Fonction	Entree	Sortie	Complexite
extraire_label_depuis_url(url)	str	str	O(k)
extraire_metadonnees(tree, url)	lxml, str	dict	O(n)
extraire_texte_article(tree)	lxml	str	O(n)
scraper_section(url, label, nb)	str, str, int	list[dict]	O(nb*T)
sauvegarder_csv(donnees, fichier)	list, str	fichier CSV	O(n)
afficher_statistiques(fichier)	str	terminal	O(n)

Pseudo-code : extraire_texte_article(tree)

```
FONCTION extraire_texte_article(tree): // Strategie 1 : cibler la balise
paragraphes <- tree.xpath("//article//p//text()") // Strategie 2 : fallback sur
```

```
classes CSS SI paragraphes est vide: paragraphes <-
tree.xpath("...//*[class='article/content']//p//text()") // Strategie 3 : dernier
recours SI paragraphes est vide: paragraphes <- tree.xpath("//p//text()") texte <-
JOINTURE(paragraphes, separateur='\n') // Nettoyage : ignorer les lignes < 20
caracteres lignes_propres <- [] POUR CHAQUE ligne DANS texte.splitlines(): SI
len(ligne.strip()) > 20: AJOUTER ligne.strip() A lignes_propres RETOURNER
JOINTURE(lignes_propres, separateur='\n') FIN FONCTION
```

PARTIE 2 — CLASSIFICATION CAMEMBERT

Pipeline de classification

Le modele CamemBERT (camembert-base, ~110M parametres) est utilise avec une tete de classification lineaire pour predire la categorie d'un article. L'entier du corpus (texte propre grace a la correction Partie 1) est utilise pour l'entrainement et l'evaluation.

Structures de donnees — Partie 2

Etape	Structure	Type	Description
Chargement CSV	DataFrame	pandas.DataFrame	50 lignes, 6 colonnes
Labels encodes	Vecteur	list[int]	economie=0, politique=1, ...
Texte tokenise	Dict tenseurs	{input_ids, attention_mask}	max_length=128
input_ids	Tenseur	torch.Tensor(1, 128)	Indices tokens vocabulaire
attention_mask	Tenseur	torch.Tensor(1, 128)	1=token reel, 0=padding
Sortie modele	Logits	torch.Tensor(batch, 5)	Score brut par classe
Prediction finale	Entier	int -> str	argmax(logits) + LabelEncoder

Pseudo-code : predire(texte)

```
FONCTION predire(texte): // Etape 1 : tokenisation tokens <- tokenizer(texte,
max_length=128, truncation=True) // tokens = {input_ids: Tensor(1,128),
attention_mask: Tensor(1,128)} // Etape 2 : passage dans CamemBERT sortie <-
CamemBERT(input_ids, attention_mask) logits <- sortie.logits // Tensor(1, 5) //
Etape 3 : prediction indice <- argmax(logits, axis=1) // Classe la plus probable
label <- LabelEncoder.inverse_transform([indice]) RETOURNER label FIN FONCTION
```

Parametres d'entrainement

Parametre	Valeur
Modele	camembert-base
Nombre d'epochs	3
Batch size	8
Learning rate	2e-5
Max tokens	128
Split train/test	80% / 20%
Plateforme	Google Colab (GPU T4)